

Atividade Avaliativa Aula 18 - Construindo APIs REST com Express.js

1 Objetivo

O objetivo desta atividade é consolidar os conceitos de desenvolvimento de APIs REST utilizando **Express** e **TypeScript**.

2 Contexto da Atividade

Uma papelaria deseja desenvolver uma API para gerenciamento de produtos cadastrados no sistema. Cada produto possui:

- ID;
- Nome;
- Preço;
- Fabricante;
- Endereço do fabricante.

O fabricante também possui uma estrutura própria contendo:

- Nome do fabricante;
- Cidade;
- País.

Inicialmente, os dados serão armazenados apenas em memória utilizando arrays.

3 Estrutura Esperada do Produto

Todos os produtos cadastrados deverão seguir a seguinte estrutura:

```
{
  "id": 1,
  "nome": "Caderno",
  "preco": 12.99,
  "fabricante": {
    "nome": "Tilibra",
    "endereco": {
      "cidade": "Sao Paulo",
      "pais": "Brasil"
    }
  }
}
```

4 Código de Referência (aula_18)

Utilize como base o código desenvolvido em sala de aula, disponível no arquivo `aula_18.ts`.

Trecho principal:

```
1  const produtos: Produto[] = [];  
2  
3  function novoProduto(req: Request, res: Response): void {  
4  
5      try {  
6  
7          let data: any = req.body;  
8  
9          if (  
10             !data.nome ||  
11             !data.preco ||  
12             !data.fabricante  
13         ) {  
14             throw new Error(  
15                 "Produto requer nome, preco e fabricante"  
16             );  
17         }  
18  
19         const produto = new Produto(  
20             data.id,  
21             data.nome,  
22             data.preco,  
23             data.fabricante  
24         );  
25  
26         // Implementar persistencia  
27  
28         res.status(200).json(produto);  
29  
30     } catch (e: unknown) {  
31  
32         res.status(400).json({  
33             Message: (e as Error).message  
34         });  
35     }  
36 }  
37 }
```

5 Tarefas

Implemente as funcionalidades necessárias para transformar o sistema em uma API CRUD completa.

1. Cadastrar Produto (POST)

Altere a função `novoProduto` para armazenar o produto no array `produtos`.

Utilize:

```
1  produtos.push(produto);  
2
```

2. Listar Produtos (GET)

Crie uma rota que retorne todos os produtos cadastrados.

3. Buscar Produto por ID (GET)

Crie uma rota que receba um ID pela URL e retorne o produto correspondente utilizando:

```
.find()
```

Caso o produto não exista, retornar:

```
status 404
```

4. Atualizar Produto (PUT)

Crie uma rota que:

- Receba o ID pela URL;
- Receba novos dados pelo body;
- Atualize as informações do produto;
- Permita alterar dados do fabricante e endereço.

5. Remover Produto (DELETE)

Crie uma rota que remova um produto do array utilizando o ID informado.

6 Regras da Atividade

- Todos os retornos devem ser em JSON;
- Utilize TypeScript corretamente;
- Utilize tipagem nas funções;
- Utilize os métodos HTTP adequados;
- Realize tratamento de erros;
- Não utilizar banco de dados;
- Os dados devem permanecer apenas em memória.

7 Códigos HTTP Esperados

Código	Descrição
200	Operação realizada com sucesso
400	Dados inválidos enviados pelo usuário
404	Produto não encontrado
500	Erro interno da aplicação

8 Desafio Extra

Implemente validações adicionais:

- Não permitir IDs duplicados;
- Validar se o preço é maior que zero;
- Validar se cidade e país foram preenchidos;
- Validar se o fabricante possui nome.

9 Exemplo de JSON para Testes

```
{
  "id": 2,
  "nome": "Caneta",
  "preco": 5.50,
  "fabricante": {
    "nome": "BIC",
    "endereco": {
      "cidade": "Manaus",
      "pais": "Brasil"
    }
  }
}
```

10 Critérios de Avaliação

- Funcionamento correto das rotas;
- Uso correto dos métodos HTTP;
- Organização do código;
- Tratamento de erros;
- Uso adequado de TypeScript;
- Manipulação correta de objetos aninhados;
- Implementação completa do CRUD.

11 Entrega

A atividade deverá ser entregue exclusivamente pelo Moodle até o dia **14/05 às 18:00**.

Link para envio:

<https://sistemas.btv.ifsp.edu.br/moodle/mod/assign/view.php?id=62505>

O aluno deverá enviar:

- Link do repositório Git contendo o projeto completo;
- Collection do Postman utilizada para testar as rotas da API;
- Código-fonte desenvolvido em TypeScript.

Importante:

- O repositório deve estar acessível ao professor;
- O projeto deve conter todas as dependências necessárias;
- A collection do Postman deve permitir testar todas as operações do CRUD;
- Entregas fora do prazo poderão sofrer desconto na nota.